

CSCI3150 Introduction to Operating Systems

Lecture 14: File Systems Part II: API, A Simple FS

Hong Xu

<https://github.com/henryhxu/CSCI3150>

Files and Directories: API

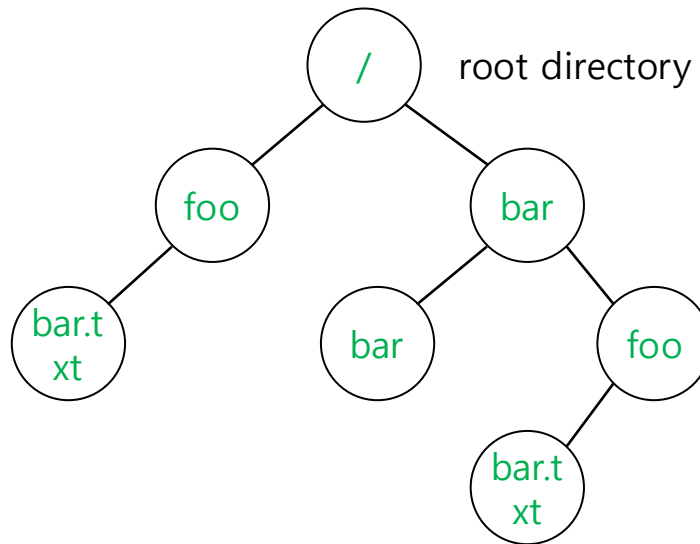
Abstractions

□ File

- ◆ A file is data with some properties (name, type, permissions, etc.)
- ◆ **inode**, "index node", a data structure that describes a FS object
- ◆ Each file has a low-level name as an 'inode number'

□ Directory

- ◆ A file; a list of <user-readable filename, low-level name> pairs



An Example Directory Tree

vaild files:

/foo/bar.txt
/bar/foo/bar.txt

vaild directories:

/
/foo
/bar
/bar/bar
/bar/foo/

File System Interface: Creating a file

- Use `open` system call with `O_CREAT` flag

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
```

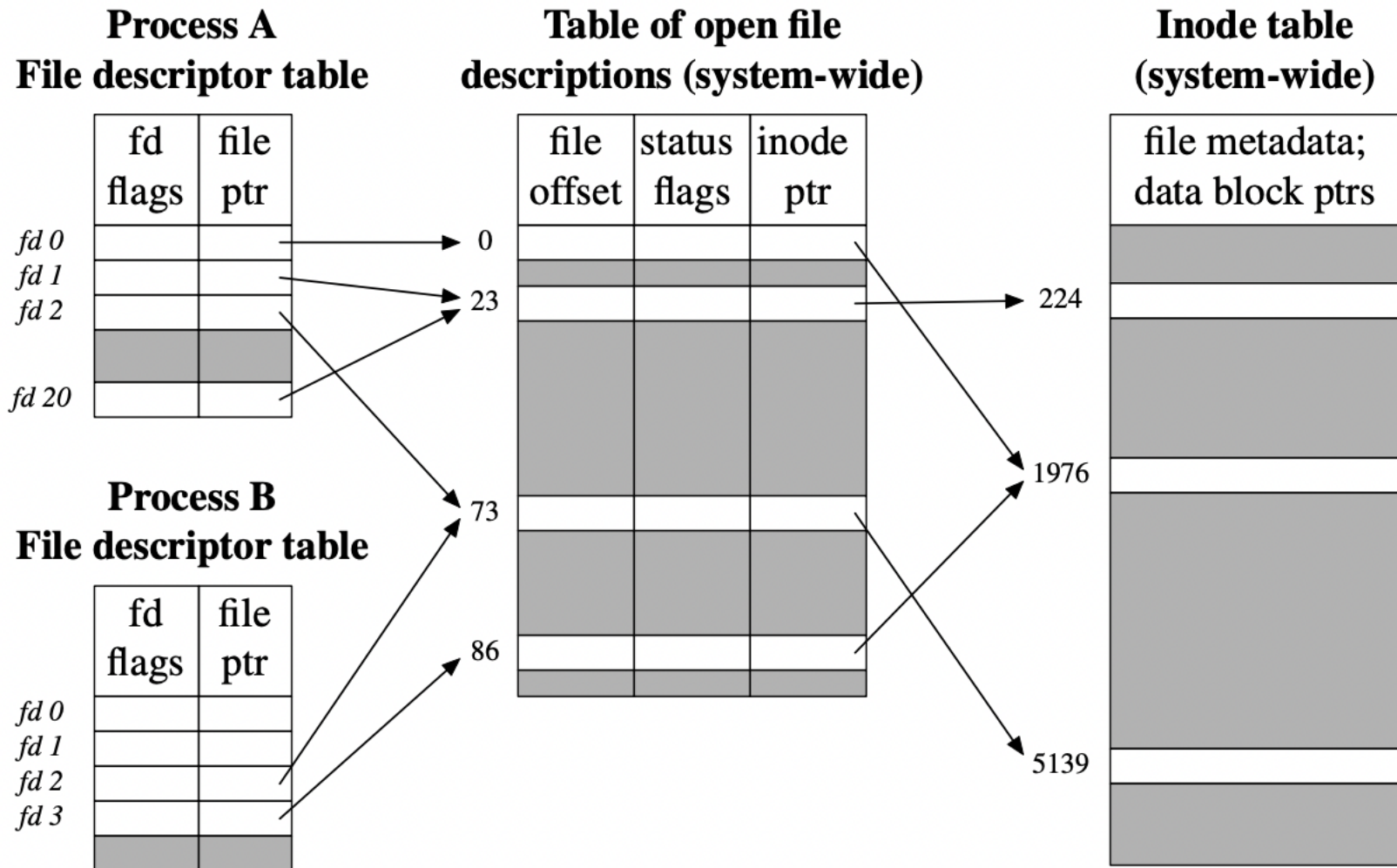
- ◆ `O_CREAT` : create a file.
- ◆ `O_WRONLY` : only write to that file while opened.
- ◆ `O_TRUNC` : make the file size zero (remove existing content).

- `open` system call returns a `file descriptor, fd`

- ◆ `fd` is an integer, and is used to access files.
- ◆ Eg: `read (file descriptor)`
- ◆ **File descriptor table**

```
struct proc {  
    ...  
    struct file *ofile[NOFILE]; // Open files  
    ...  
};
```

Aside: A Few Tables for Files in Unix/Linux

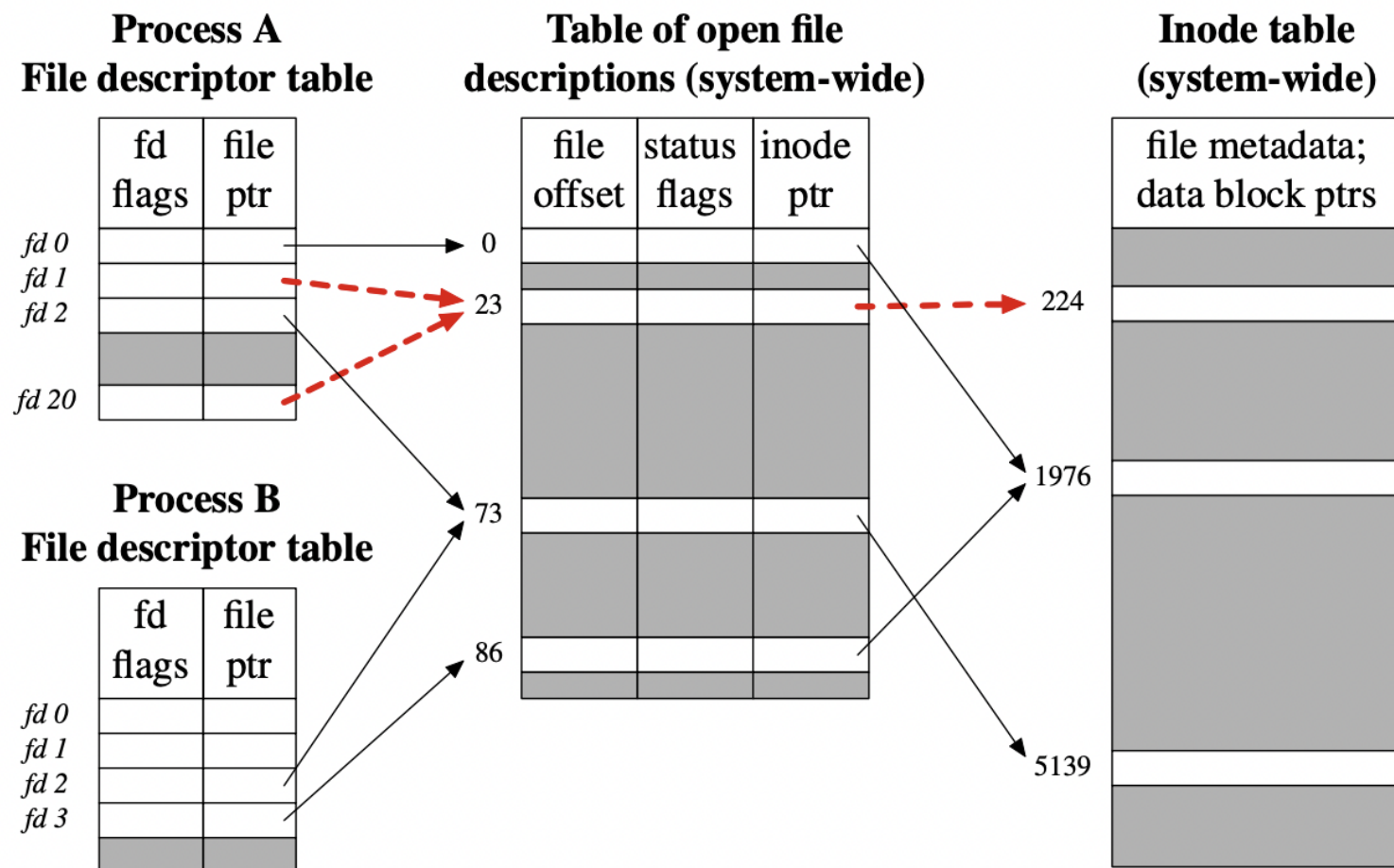


Aside: A Few Tables for Files in Unix/Linux

- ▣ File descriptor table
 - ◆ **Per-process**, one entry for each fd opened by this process
 - ◆ Flags controlling the operations of a fd
- ▣ Open file table
 - ◆ **Global**, system-wide, one entry for each file opened by "`open()`"
 - ◆ File offset, access mode, etc.
 - ◆ Terms: Open file table entry, open file description (OFD) (POSIX)
- ▣ inode table
 - ◆ **System-wide**, one entry per inode in the fs
 - ◆ File type (regular file, socket, etc.)
 - ◆ Permissions, properties, etc.

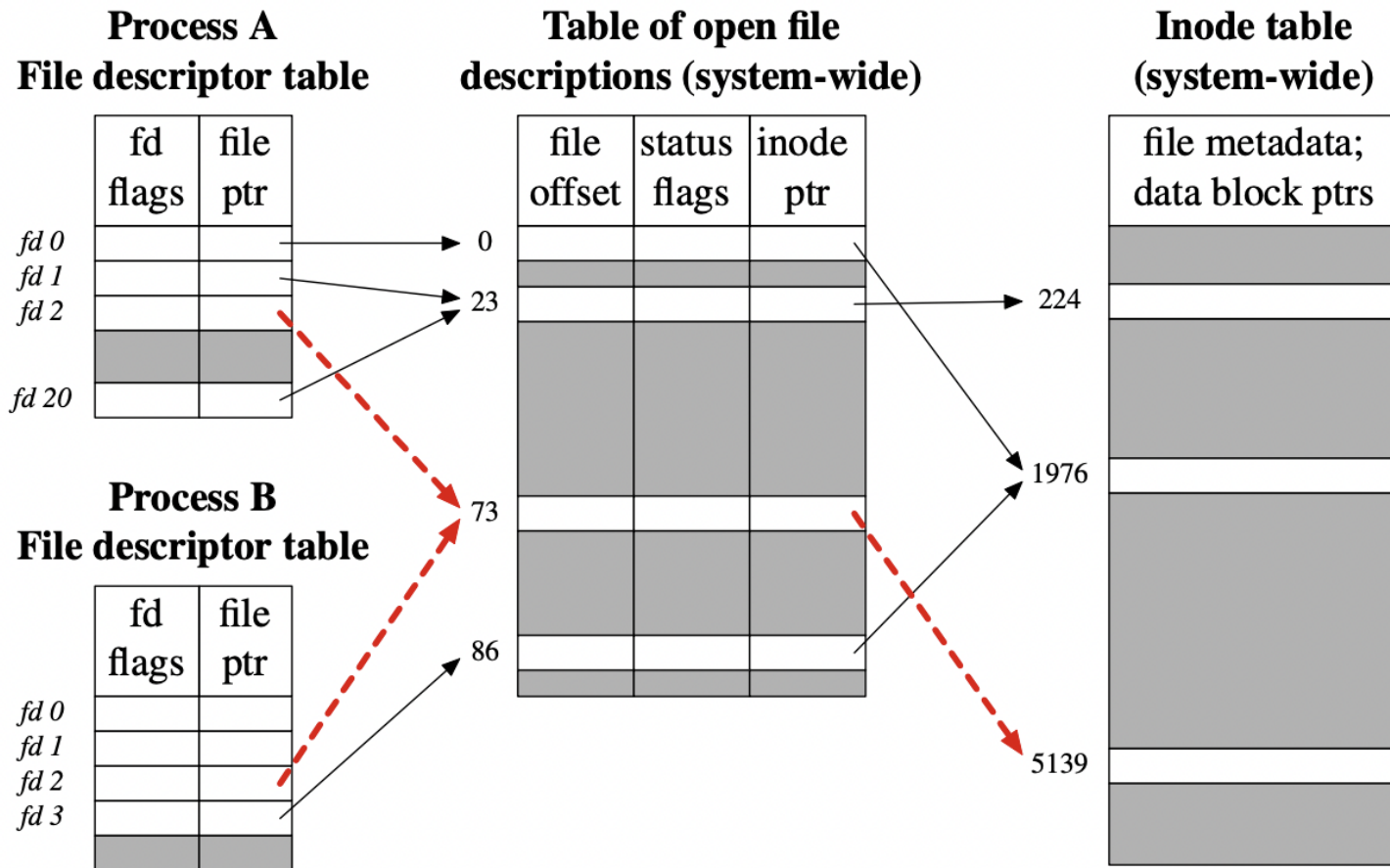
Duplicated File Descriptors (intra-process)

- A process may have multiple fds referring to the same OFD
 - ◆ Achieved with `dup()` or `dup2()`



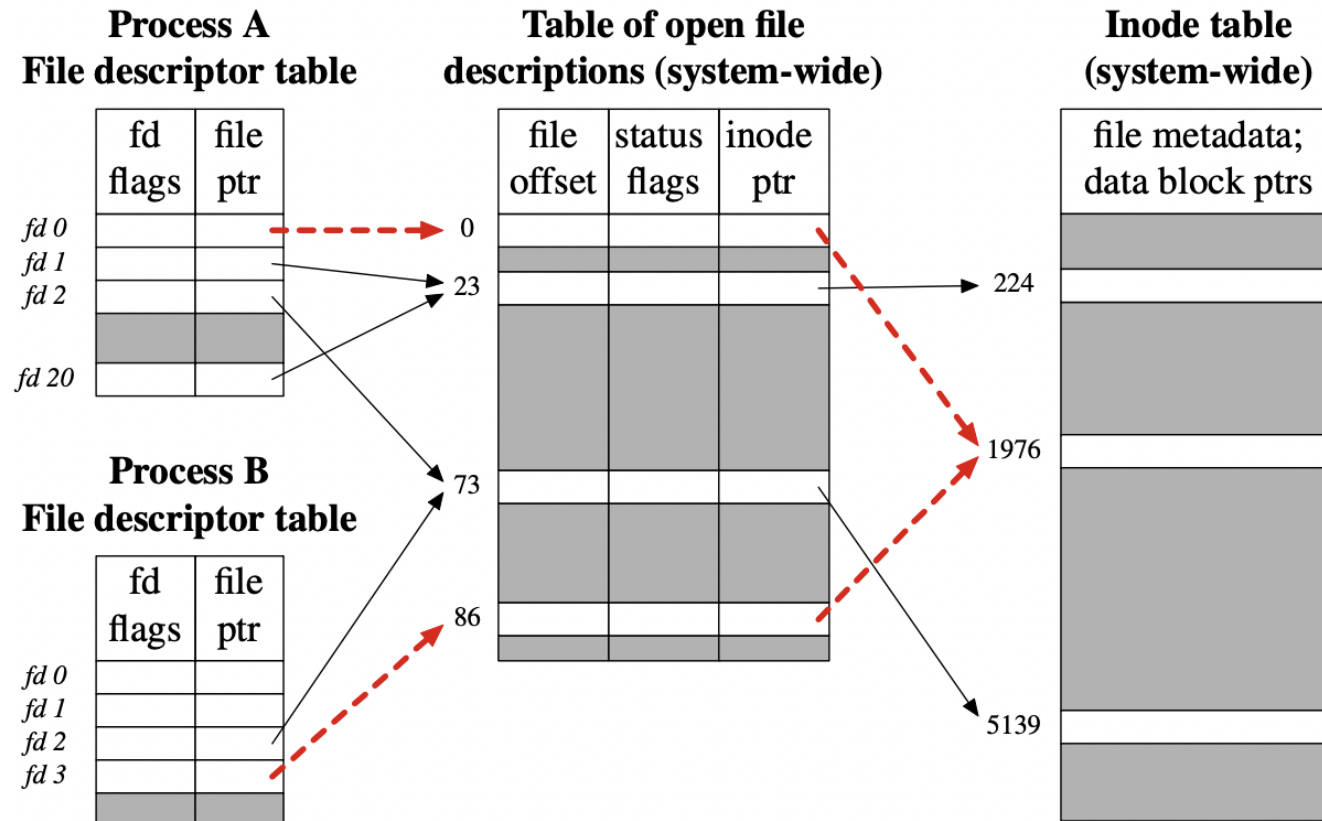
Duplicated File Descriptors (inter-process)

- Two processes may have fds referring to the same OFD
 - As a result of `fork()`



Duplicated OFDs

- Two processes may have fds referring to distinct OFDs, or open file table entries, that refer to the same inode
 - They independently opened the same file



Why so complicated? 🤔

- ❑ Dup. OFDs to the same inode: Allow a file to be read by multiple programs at the same time
- ❑ Dup FDs to the same OFD:

1. Sequential Coordination (The "Pipeline" Semantics)

This is the most common reason. If two parts of a program (or two processes) need to read a file **sequentially** without stepping on each other's toes, they share an OFD.

- **Scenario:** You want to read a log file in chunks.
- **If separate OFDs:** Both start at offset 0. You read the same lines twice.
- **If shared OFD (via `dup` or `fork`):**
 - FD A reads 100 bytes → Offset advances to 100.
 - FD B reads next → Starts at 100.

Interface: Reading and Writing Files

- An Example of reading and writing 'foo' file.

```
prompt> echo hello > foo //save the output to the file foo
prompt> cat foo          //dump the contents to the screen
hello
prompt>
```

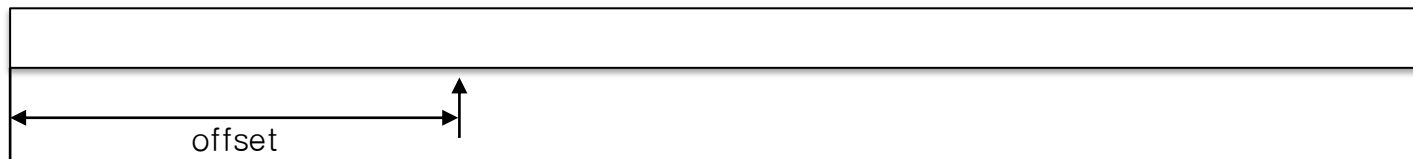
- The result of `strace` to figure out what `cat` is doing

```
prompt> strace cat foo //strace to figure out what cat is doing
...
open("foo", O_RDONLY|O_LARGEFILE) = 3
read(3, "hello\n", 4096)           = 6 /*the number of bytes to read*/
write(1, "hello\n", 6)              = 6
hello
read(3, "", 4096)                  = 0
..
prompt>
```

- ◆ `open()`: open file for reading with `O_RDONLY` and `O_LARGEFILE` flags.
 - returns file descriptor 3 (0,1,2, is for standard input/output/error)
- ◆ `read()`: read bytes from the file.
- ◆ `write()`: write buffer to standard output.

Reading and Writing Files (Cont.)

▣ OFFSET



- ◆ The position of the file where we start read and write.
- ◆ When a file is open, "an offset" is allocated.
- ◆ Updated after read/write

▣ How to read or write to a specific offset within a file ?

```
off_t lseek(int fd, off_t offset /*location */, int whence);
```

- ◆ Third argument is how the seek is performed.
 - `SEEK_SET` : to offset bytes.
 - `SEEK_CUR`: to its current location plus offset bytes.
 - `SEEK_END`: to the size of the file plus offset bytes.

Data structures

```
struct file {  
    int ref;  
    char readable;  
    char writable;  
    struct inode *ip;  
    uint off;  
};
```

```
struct {  
    struct spinlock lock;  
    struct file file[NFILE];  
} ftable;
```

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
read(fd, buffer, 100);	100	100
read(fd, buffer, 100);	100	200
read(fd, buffer, 100);	100	300
read(fd, buffer, 100);	0	300
close(fd);	0	—

Sample traces

System Calls	Return Code	OFT[10] Current Offset	OFT[11] Current Offset
fd1 = open("file", O_RDONLY);	3	0	–
fd2 = open("file", O_RDONLY);	4	0	0
read(fd1, buffer1, 100);	100	100	0
read(fd2, buffer2, 100);	100	100	100
close(fd1);	0	–	100
close(fd2);	0	–	–

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
lseek(fd, 200, SEEK_SET);	200	200
read(fd, buffer, 50);	50	250
close(fd);	0	–

`fsync()`

▣ Persistency

- ◆ `write()` : write data to the buffer. Later, save it to the storage.
- ◆ some applications require more than eventual guarantee. Ex) DBMS

▣ `fsync()` : the writes are forced immediately to disk.

```
off_t fsync(int fd /*for the file referred to by the specified file*/)
```

▣ An Example of `fsync()`.

```
1  int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);  
2  int rc = write(fd, buffer, size);  
3  rc = fsync(fd);
```

- ◆ If a file is created, it needs to be durably a part of the directory.
 - Above code requires `fsync()` to directory also.

A Simple File System

Overview

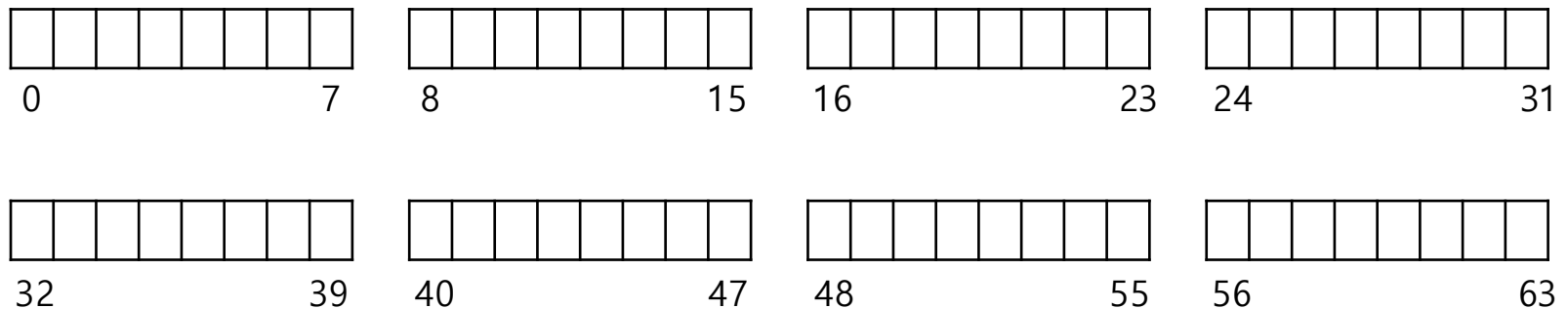
- ▣ We will study a very simple file system (vsfs)
 - ◆ Basic on-disk structures, access methods, and various policies of fs
- ▣ We will study...
 - ◆ How can we build a simple file system?
 - ◆ What structures are needed on the disk?
 - ◆ What do they need to track?
 - ◆ How are they accessed?

File System Implementation

- ▣ What types of **data structures** are utilized by the file system?
- ▣ How does a file system organize its data and metadata?
- ▣ Understand access methods of a file system.
 - ◆ `open()`, `read()`, `write()`, etc.

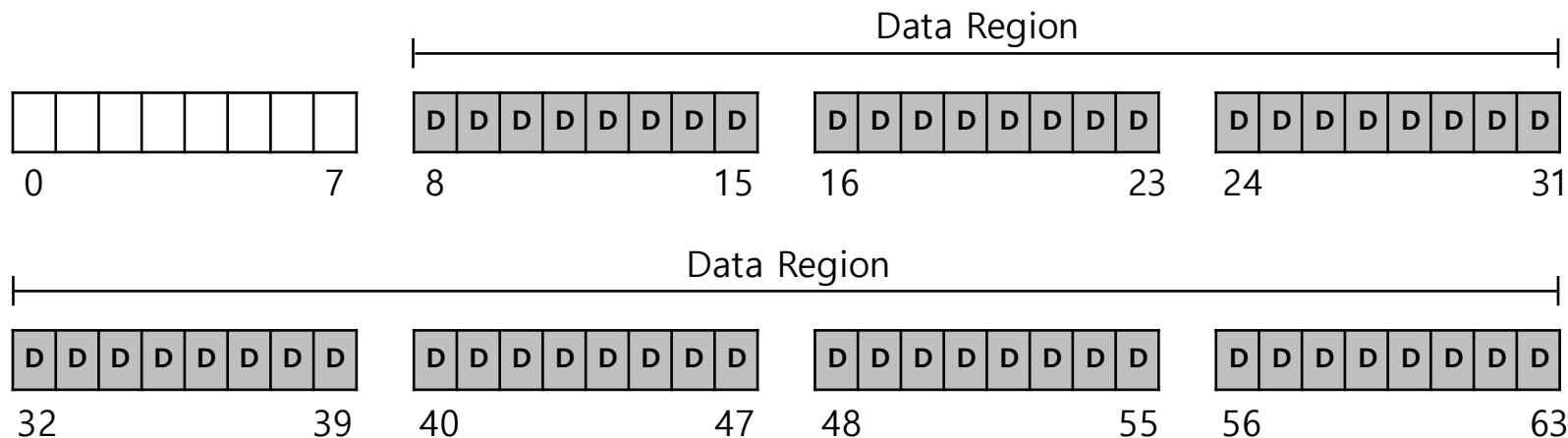
Overall Organization

- ▣ Let's develop the overall organization of the file system data structure.
- ▣ Divide the disk into **blocks**.
 - ◆ Block size is 4 KB.
 - ◆ The blocks are addressed from 0 to $N - 1$.



Data Region in a FS

- Reserve a **data region** to store user data



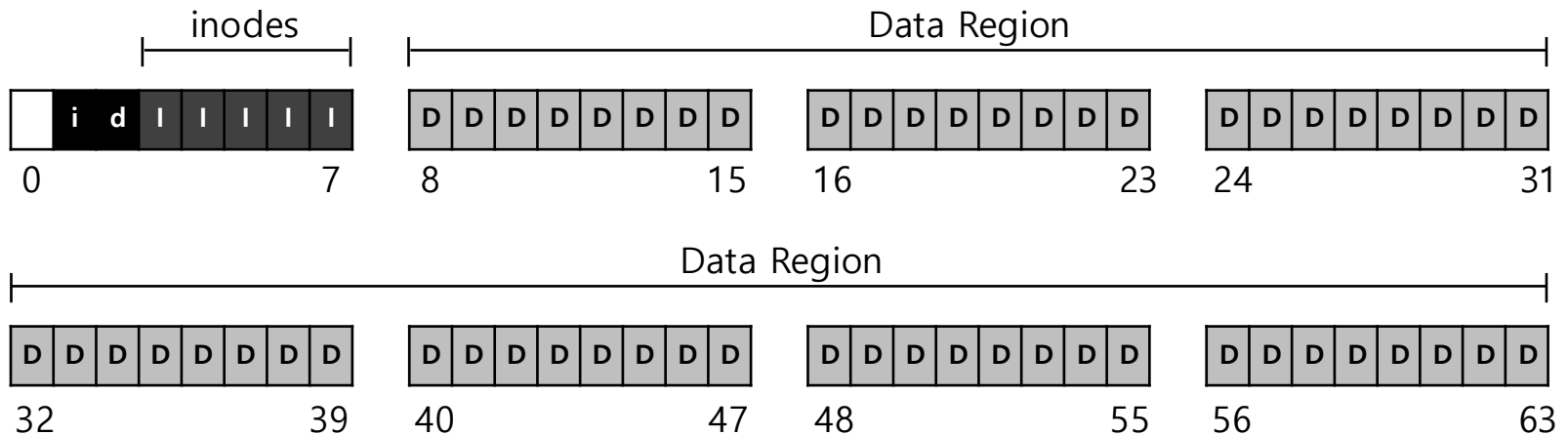
- The FS has to track which data blocks comprise a file, the size of the file, its owner, etc.

How we store these inodes in file system?

inode Table

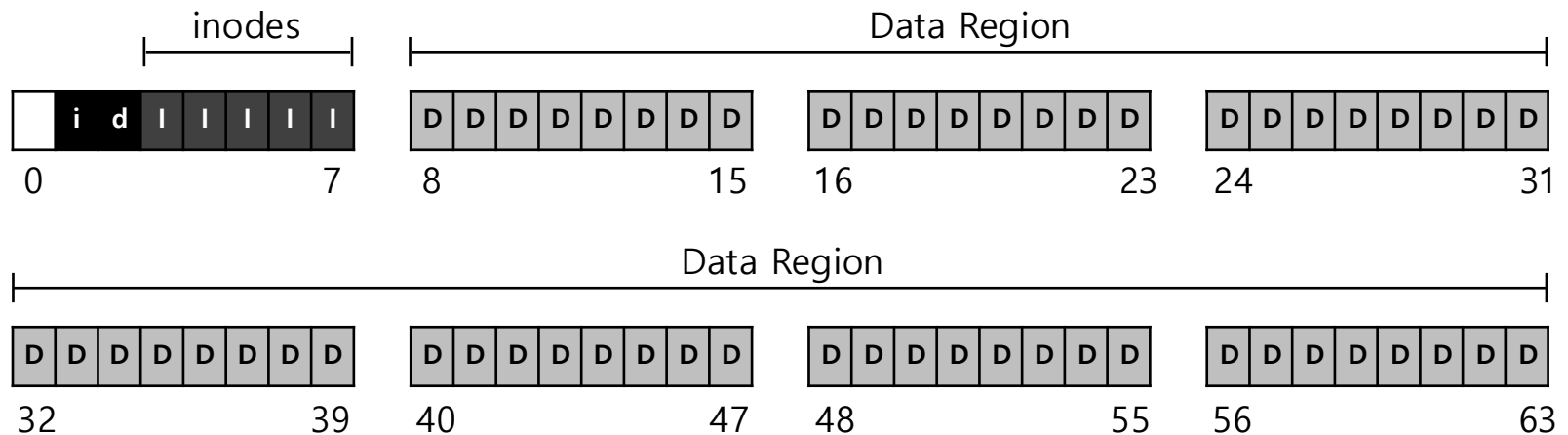
▣ Reserve some space for **inode table**

- ◆ This holds an array of on-disk inodes
- ◆ Ex) inode tables: 3 ~ 7, inode size: 256B
 - A 4KB block can hold 16 inodes
 - The file system contains 80 inodes (maximum number of files)



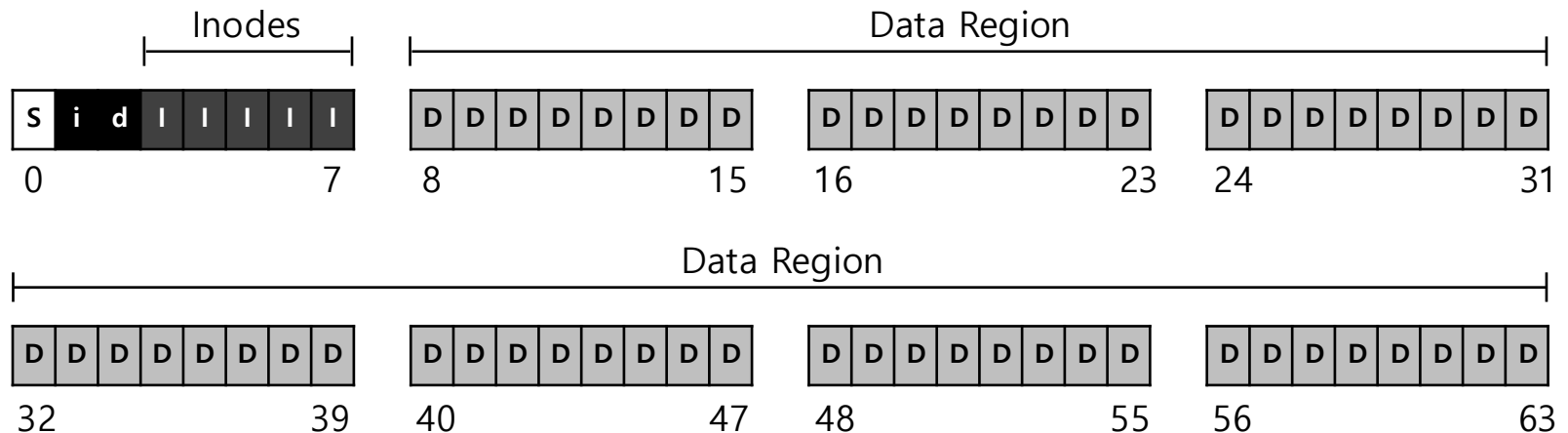
Allocation Structures

- ▣ This is to track whether inodes or data blocks are free or not.
- ▣ Use a **bitmap**, each bit indicates free (0) or in-use (1)
 - ◆ data bitmap: for data region
 - ◆ inode bitmap: for inode table



Super Block

- Super block contains meta information for a **file system**
 - ◆ Ex) The number of inodes, begin location of inode table. etc



- Thus, when mounting a file system, OS will read the superblock first, to initialize various information.

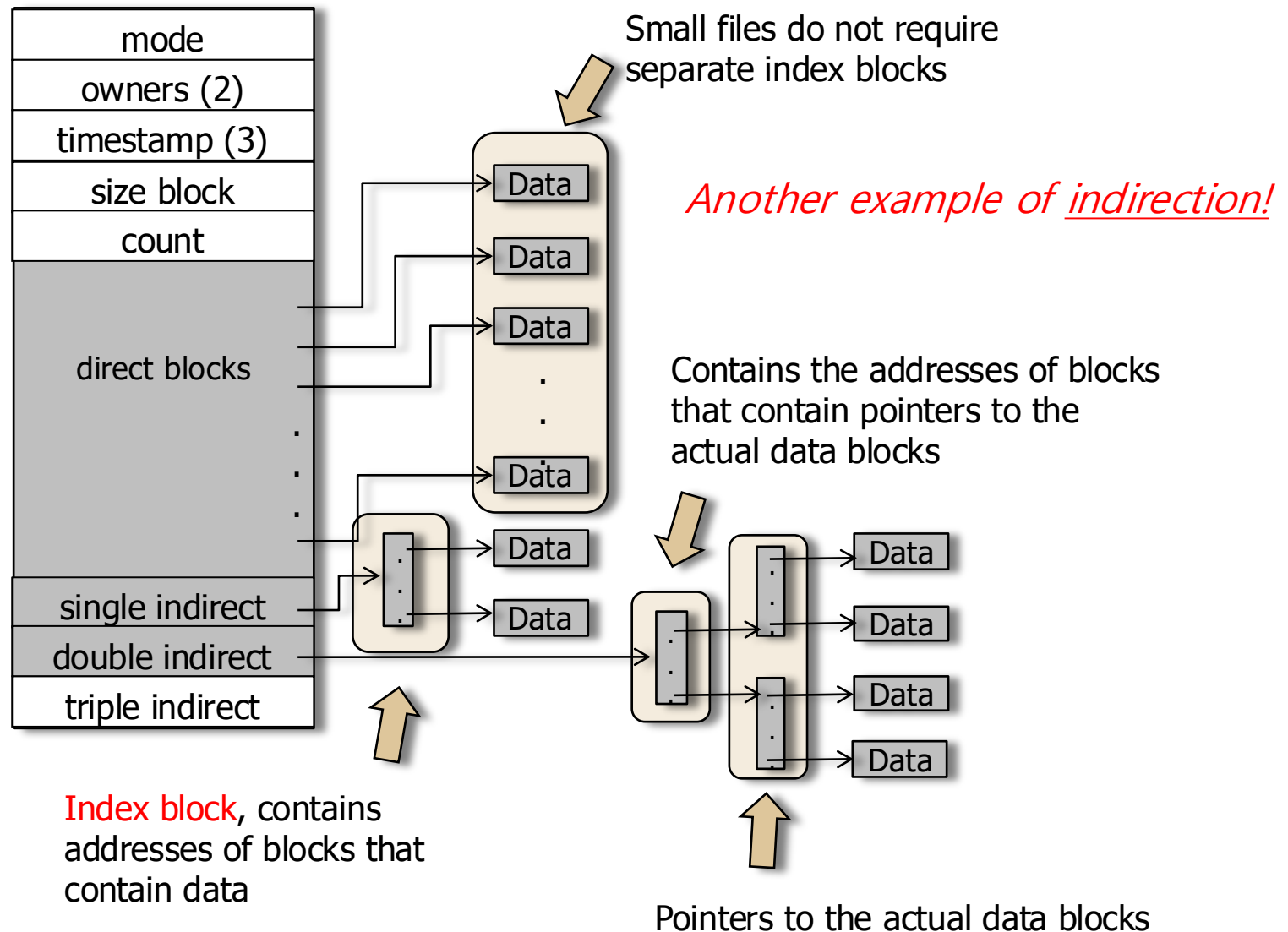
File Organization: The inode

- Each inode is referred to by an inode number.
 - by inode number, the FS calculates where the inode is on the disk.
 - Ex) inode number: 32
 - Calculate the offset into the inode region $(32 \times \text{sizeof}(\text{inode}) (256\text{B}) = 8192$
 - Add start address of the inode table(12 KB) + inode region(8 KB) = 20 KB

The inode table

				iblock 0				iblock 1				iblock 2				iblock 3				iblock 4			
Super	i-bmap	d-bmap	0	1	2	3	16	17	18	19	32	33	34	35	48	49	50	51	64	65	66	67	
			4	5	6	7	20	21	22	23	36	37	38	39	52	53	54	55	68	69	70	71	
			8	9	10	11	24	25	26	27	40	41	42	43	56	57	58	59	72	73	74	75	
			12	13	14	15	28	29	30	31	44	45	46	47	60	61	62	63	76	77	78	79	
0KB	4KB	8KB	12KB	16KB	20KB	24KB	28KB	32KB															

inode Structure: Indexed Allocation



Directory Structure

Record length		String length	
inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	7	foobar

File Read: "/foo/bar"

Assumption: Super node is in memory; everything else is not

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read	read	read	read	read			
read()					read			read		
read()					read				read	
read()					read					read

To find an inode, we need the i-number which usually is in its parent directory
The root has no parent; its i-number must be well-known (2 in most Unix FS)

File Creation

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		read write	read	read	read write	read	read write			
write()	read write			write	read write			write		

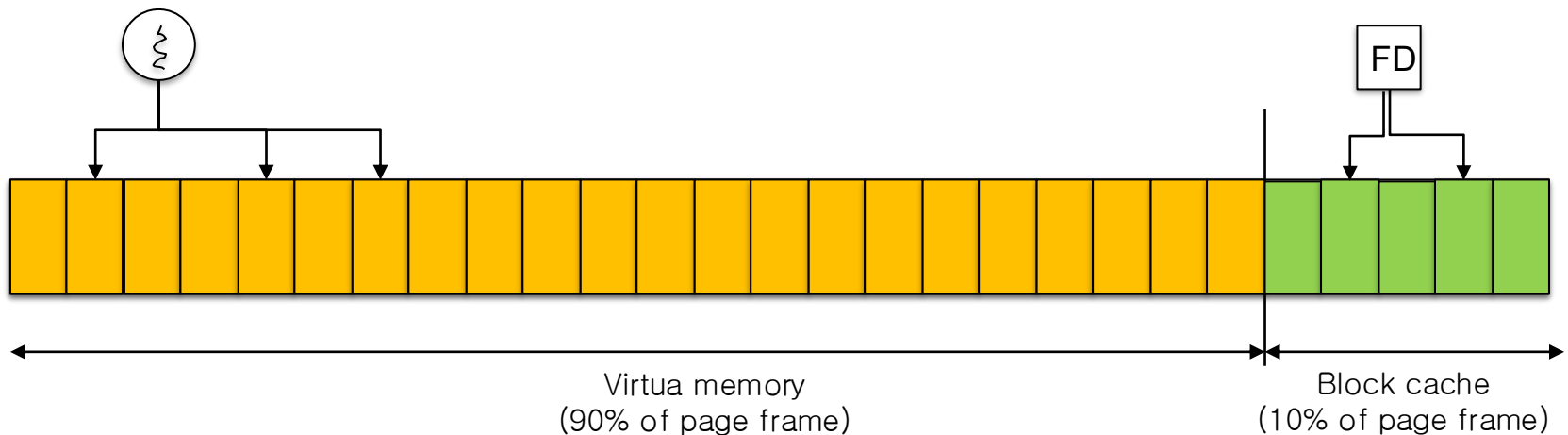
...

File Creation (Cont.)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
					...					
write()	read write				read write				write	
write()	read write				read write					write

Caching and Buffering

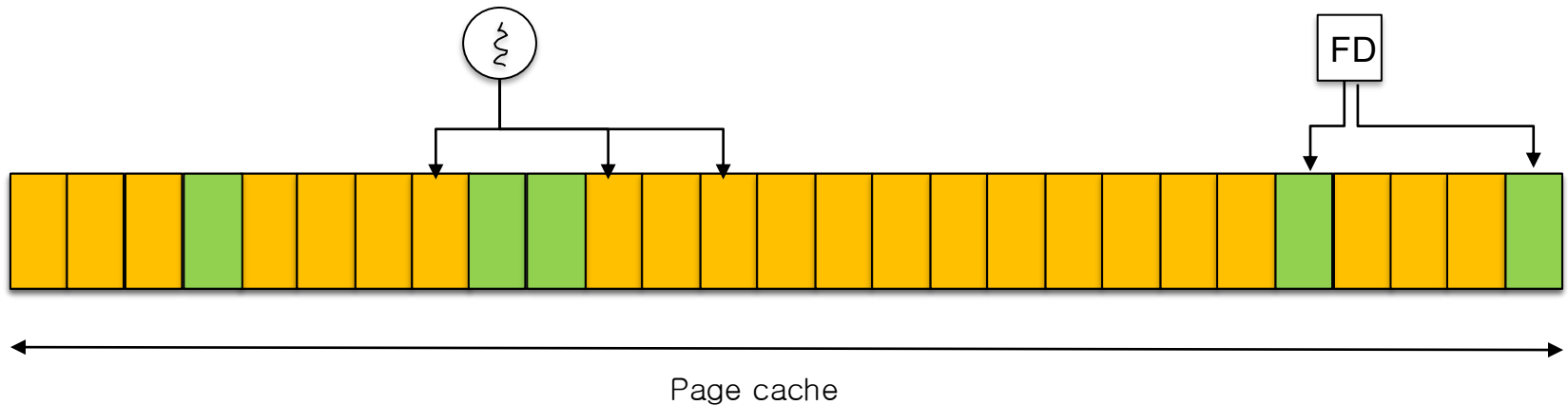
- ▣ Reading and writing are very IO-intensive
 - ◆ File open: two IOs for each directory component and one read for the data.
- ▣ Cache popular blocks
 - ◆ Reduce the IO
 - ◆ LRU or others as the replacement policy
 - ◆ Static partitioning: 10% of DRAM, but very inefficient



Caching and Buffering

□ Page Cache

- ◆ Merge virtual memory and buffer cache
- ◆ A physical page frame can host either a page in the process address space or a file block.
- ◆ Dynamic partitioning



Summary

- ▣ Requirements for building filesystem
 - ◆ File information: inode
 - ◆ File structure: indexed file
 - ◆ Directory (name→inode-number): array of <inode #, name>'s
 - ◆ Free block information: Bitmap

How to Make it Faster?

Problem of the VSFS

- ❑ VSFS, or Unix file system, treats the disk as a **random-access memory**
- ❑ Example of random-access blocks with four files
 - ◆ Data blocks for each file can be accessed by going back and forth on the disk

A1	A2	B1	B2	C1	C2	D1	D2
----	----	----	----	----	----	----	----

- ◆ File b and d is deleted.

A1	A2			C1	C2		
----	----	--	--	----	----	--	--

- ◆ File E is created with free blocks. (**spread across** the disk)

A1	A2	E1	E2	C1	C2	E3	E4
----	----	----	----	----	----	----	----

FFS: Disk awareness is the solution

- ▣ FFS is **Fast File System** designed by a group at Berkeley
- ▣ The key idea of FFS is that file system structures and allocation policies need to be “disk aware” to improve performance
 - ◆ Keep same API with file system. (`open()`, `read()`, `write()`, etc)
 - ◆ Change the internal implementation

Cylinder Groups

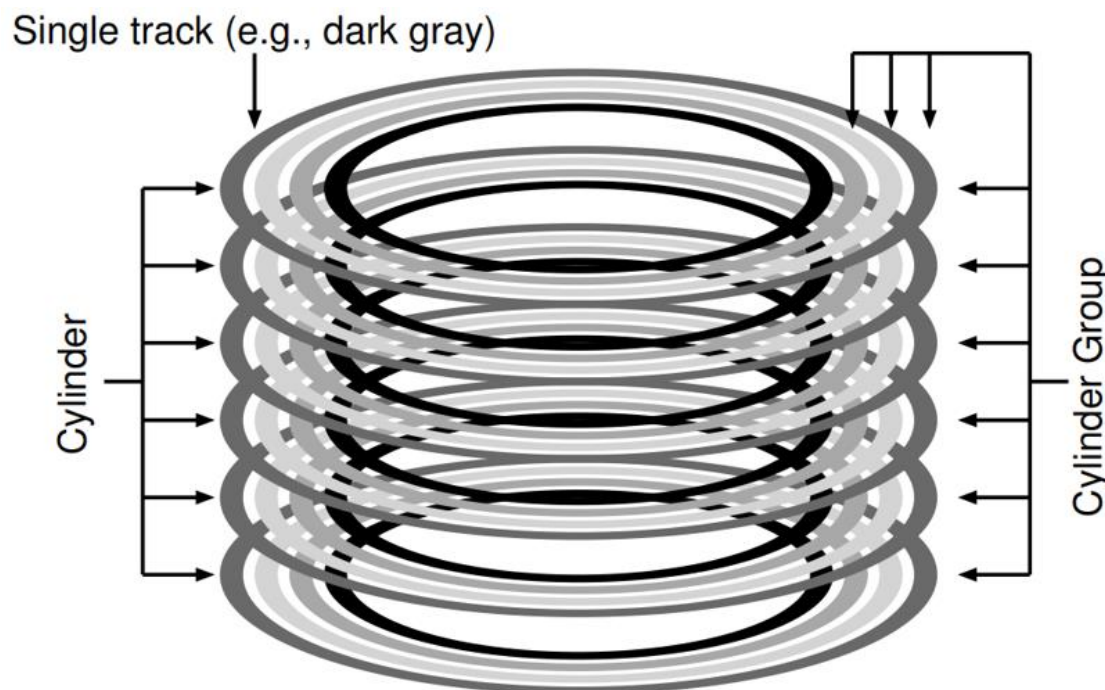
- ▣ FFS divides the disk into a bunch of groups, **Cylinder Groups**
 - ◆ Modern file system call cylinder groups as *block groups*
- ▣ These groups are used to improve seek performance.
 - ◆ By placing two files within the same group.
 - ◆ Accessing one after the other **will not be long seeks** across the disk.
 - ◆ FFS needs to allocate the files and directories within each of these groups.



- ▣ Data structure for each cylinder group.
 - ◆ A copy of the super block for reliability reason.
 - ◆ inode bitmap and data bitmap to track free inode and data block.
 - ◆ inodes and data block

Cylinder Group (Cont.)

- ▣ **Cylinder:** Tracks at same distance from center of drive across different surfaces.
 - ◆ All tracks with same color
- ▣ **Cylinder Group:** Set of N consecutive cylinders
 - ◆ if $N=3$, first group does not include black track



How To Allocate Files and Directories?

- ▣ Policy is “**keep related stuff together**”
- ▣ The placement of directories
 - ◆ Find the cylinder group with a low number of allocated directories and a high number of free inodes.
 - ◆ Put the directory data and inode in that group.
- ▣ The placement of files.
 - ◆ Allocate data blocks of a file in the same group as its inode
 - ◆ It places all files in the same group as their directory

Summary

- ▣ The introduction of fast file system (FFS)
 - ◆ It makes the file system fast, considering characteristics of a disk
- ▣ Many file systems take cues from FFS
 - ◆ E.g. ext4, ext3, ext4